

TALENTHUB WEB PLATFORM

Frontend Development SOP

VERSION	1.0
DATE	20 May 2026
STACK	Vite 7 · React 19 · Tailwind v4 · Redux Toolkit · axios · react-hook-form + zod · Playwright
AUDIENCE	Frontend engineers (junior → senior) using AI-augmented tooling

INTERNAL ENGINEERING DOCUMENT · V1.0

Table of Contents

1. Project Overview & Architecture
 2. Development Environment Setup
 3. Directory Structure & Naming Conventions
 4. Design System & Theming Reference (Tailwind v4 + @theme)
 5. Component Development Standards
 6. State Management with Redux Toolkit
 7. API Integration with axios
 8. Forms with react-hook-form + zod
 9. Routing & Layout Architecture
 10. Page Development Procedures
 11. Wiki Discipline (the source of truth)
 12. Debug-Log Discipline (mandatory)
 13. AI-Augmented Workflow (Claude Code + MCP)
 14. Testing & Verification
 15. Git Workflow & Pre-Commit Hooks
 16. Security & Performance Standards
 17. Accessibility Standards
 18. Code Review Framework
 19. Figma Fidelity & Design Reference
 20. Appendices
-

1. Project Overview & Architecture

1.1 What is TalentHub?

TalentHub is a Ghana-rooted talent and career-development platform. Talents discover career paths, build a portfolio, complete a guided onboarding journey, customise an in-platform avatar, earn XP / milestones, and surface themselves to recruiters and partners. A reserved `/admin/*` slot will host the back-office subsystem (lazy-loaded) when that product area lands.

The frontend in this repository ships the **talent-facing experience**:

- Landing page (`/`)
- Authentication surfaces (`/login` , `/get-started`)
- Onboarding journey (`/onboarding/talent/welcome` ... `/onboarding/talent/review`)
- Profile engagement — identity map, milestone unlocks, avatar customiser (`/profile/engagement/*`)
- A design-system playground (`/components`)

1.2 Two Chrome Contexts

The platform has two distinct UI shells, driven by route inside `MainLayout` :

- **Main chrome** — full marketing `Navbar` + page + full `Footer` (logo, link columns, contact). Used by the landing, design-system, and post-onboarding surfaces.
- **Onboarding chrome** — slim cream `OnboardingNavbar` + page + **slim onboarding footer** (per Figma, the onboarding flow keeps a minimal footer — typically copyright + a short legal link row — so the user retains context without being pulled into the marketing nav). Used by `/login` , `/get-started` , and every `/onboarding/*` page.

⚠ **Known drift (as of 2026-05-20):** The current `MainLayout.jsx` renders **no footer at all** on onboarding paths (`{onboarding ? null : <Footer />}`), based on an older reading of the design. Figma actually shows a slim onboarding footer. Track the fix under a small `feat(layout): slim onboarding footer` task, update the relevant wiki claim from `✅ VERIFIED` to `❌ STALE` until the slim footer ships, then re-verify.

Profile engagement pages (`/profile/engagement/*`) own their own chrome and mount **outside** `MainLayout` .

1.3 Technology Stack

Layer	Tech	Version	Purpose
Build	Vite	^7.0.4	Dev server + production build
UI	React	^19.2.0	Function components + hooks only
Language	Plain JavaScript	—	No TypeScript
Routing	React Router DOM	^6.30.1	Config-based routing in <code>src/App.jsx</code>

Layer	Tech	Version	Purpose
State	Redux Toolkit	^2.11.2	Global state via slices
State (persist)	redux-persist	^6.0.0	Auth + profile persistence (wire when auth lands)
HTTP	axios	^1.12.2	Single configured instance in <code>src/services/api.js</code>
Forms	react-hook-form	^7.54.0	Form state + validation orchestration
Validation	zod	^3.24.1	Schema-first validation
Form bridge	@hookform/resolvers	^3.9.1	Connects zod to react-hook-form
Styling	Tailwind CSS	^4.1.11	Utility-first; tokens in <code>src/index.css</code> <code>@theme</code>
Cookies	universal-cookie	^8.0.1	Cookie read/write (auth-adjacent surfaces)
Lint	ESLint flat config	^9.30.1	Code quality
Format	Prettier	^3.4.2	Code style
E2E	Playwright	^1.49.1	End-to-end tests in <code>tests/e2e/</code>
Hooks	Husky + lint-staged	^9.1.7 / ^15.3.0	Pre-commit + commit-msg hooks
Commits	commitlint conventional	^19.6.x	Conventional-commit enforcement

1.4 Architecture (Mental Model)

```

Browser
├─ React App (Vite dev server / static build)
│  └─ BrowserRouter (src/App.jsx)
│     └─ MainLayout (Navbar | OnboardingNavbar + <Outlet/> + Footer?)
│        ├── LandingPage      /
│        ├── LoginPage        /login
│        ├── GetStartedPage    /get-started
│        ├── Onboarding flow   /onboarding/talent/* (wrapped in OnboardingProvider)
│        └─ HomePage (DS playground) /components
│  └─ Profile engagement (own chrome) /profile/engagement/*
├─ Redux store (src/store/index.js)
│  └─ Slices (src/store/slices/* – add as features land)
│     └─ Thunks → axios (src/services/api.js) → Backend API
├─ Tailwind v4 (@tailwindcss/vite)
│  └─ Design tokens (src/index.css @theme block)

```

1.5 Design Viewport

Figma designs live at the **1728 px desktop frame**. The `MainLayout` clamps the rendered tree to `max-w-[1728px]` and centres it on larger viewports. Mobile + tablet frames are **derived via `clamp()`** rather than separate static breakpoints — see §19 (Figma Fidelity) and the rules in `wiki/figma-fidelity.md`.



2. Development Environment Setup

2.1 Prerequisites

Required:

- **Node.js** ≥ 18 (`node -v`) — `package.json` `engines` enforces this
- **npm** ≥ 9 (`npm -v`)
- **Git** (`git --version`)

Recommended:

- **VS Code** with extensions: ESLint, Prettier, Tailwind CSS IntelliSense, ES7+ React/Redux snippets
- **Playwright extension** (`ms-playwright.playwright`) for spec running from the IDE
- **Figma desktop app** (required for Figma Dev Mode MCP — see §13.4)

2.2 Clone and Install

```
git clone <repository-url> talenthub-frontend
cd talenthub-frontend
npm install # installs deps; husky prepares git hooks via "prepare"
cp .env.example .env # fill in VITE_API_BASE_URL
```

2.3 Environment Variables

Vite only exposes variables prefixed with `VITE_` to the client. Read them via `import.meta.env.VITE_*`.

Variable	Required	Example	Where it's read
<code>VITE_API_BASE_URL</code>	yes	<code>https://api.talenthub.example/v1</code>	<code>src/services/api.js</code>
<code>VITE_APP_NAME</code>	no	<code>TalentHub</code>	UI strings (titles, labels)

Restart the dev server after editing `.env` — Vite reads it only at startup.

2.4 Scripts

Command	Purpose
<code>npm run dev</code>	Vite dev server on http://localhost:5173
<code>npm run build</code>	Production build to <code>dist/</code>
<code>npm run preview</code>	Serve the production build on port 4173
<code>npm run lint</code>	ESLint (flat config) over the repo
<code>npm run lint:fix</code>	ESLint with auto-fix
<code>npm run format</code>	Prettier write (whole repo)

Command	Purpose
<code>npm run format:check</code>	Prettier check (CI-safe, no writes)
<code>npm run test:e2e</code>	Playwright suite (boots dev server automatically)
<code>npm run test:e2e:ui</code>	Playwright UI mode
<code>npm run test:e2e:headed</code>	Playwright with a visible browser

2.5 Build Pipeline (how the bytes flow)

The Tailwind v4 compiler runs as a **Vite plugin** (`@tailwindcss/vite`), not via PostCSS:

- `src/index.css` imports Tailwind and declares the `@theme` block — the **single source of truth** for design tokens (colors, typography, radii, shadows, spacing).
- `@tailwindcss/vite` reads the `@theme` block and generates every utility class (`bg-brand-green` , `text-display-md` , `rounded-pill` , ...).
- `tailwind.config.js` is *informational only* — it exists for editor tooling. **Do not put tokens there.**
- Vite serves the result in dev with HMR and bundles a chunked production build (vendor / router / redux split per `vite.config.js`).

2.6 MCP Servers (Claude Code sessions)

The project ships an `.mcp.json` declaring two MCP servers Claude Code loads at session start:

- **Playwright MCP** (`@playwright/mcp@latest` , npx-launched) — navigate, screenshot, inspect the browser console.
- **Figma MCP** (HTTP, `http://127.0.0.1:3845/mcp`) — read Figma design specs.

The Figma server is provided by the **official Figma desktop app**. Enable it under *Figma → Preferences → Enable Dev Mode MCP Server* before starting a session that touches design extraction. If you change `.mcp.json` , restart Claude Code so the new server registers.

3. Directory Structure & Naming Conventions

3.1 Canonical Tree (current + reserved)

```

talenthub-frontend/
├── .husky/                # pre-commit + commit-msg hooks (Husky)
├── .mcp.json              # Playwright + Figma MCP for Claude Code
├── .prettierrc            # Prettier config (semi true, singleQuote, printWidth 100)
├── .editorconfig          # cross-editor LF + 2-space indent
├── eslint.config.js       # ESLint flat config + prettier compatibility
├── playwright.config.js   # Playwright (boots dev server on :5173)
├── vite.config.js         # Vite + react + @tailwindcss/vite + chunk splitting
├── tailwind.config.js     # informational only (tokens live in src/index.css)
├── commitlint.config.js   # conventional-commits
├── vercel.json            # Vercel SPA rewrites
├── CLAUDE.md              # engineering rules for Claude Code sessions
├── README.md              # quick-start
├── docs/
│   ├── SOP.md             # ← this document
│   └── user stories/      # GTH user-story DOCX bundles (per feature batch)
├── wiki/                  # living knowledge base (Karpathy LLM-Wiki pattern)
│   ├── INDEX.md
│   ├── architecture.md
│   ├── routing.md
│   ├── api.md
│   ├── components.md
│   ├── design-tokens.md
│   ├── figma-fidelity.md
│   ├── figma-node-map.md
│   ├── debugging-workflow.md
│   ├── pre-commit-workflow.md
│   └── log.md
├── public/                # static assets served as-is
├── src/
│   ├── main.jsx           # StrictMode + createRoot + index.css
│   ├── App.jsx            # BrowserRouter + Routes (config-based)
│   ├── App.css            # minimal global resets
│   ├── index.css          # * Tailwind v4 @theme tokens (design system)
│   ├── admin/             # reserved – future lazy-loaded admin subsystem
│   ├── assets/
│   │   ├── brand/         # logo PNG / SVG
│   │   ├── hero/          # landing-page hero assets
│   │   ├── login/         # auth-page aside imagery
│   │   ├── engagement/    # avatar + milestone assets
│   │   └── getStarted/    # role-select imagery
│   ├── components/
│   │   └── ui/             # design-system primitives (Button, Card, Modal, ...)
│   │       └── form/      # Field, TextInput, Textarea, Select, Checkbox, VerificationCode,
├── Upload
│   ├── shared/            # cross-feature (Navbar, OnboardingNavbar, Footer, Logo, modals)
│   ├── sections/         # per-page composed sections (e.g. landing/HeroSection.jsx)
│   │   └── cards/         # specialised cards (EntryMethodCard, StageMapNode, ...)
│   ├── constants/        # ROUTES (future), STORAGE_KEYS, PROFILE_STAGES, ...
│   ├── hooks/            # custom React hooks (useOnboarding, useDebounce, ...)
│   ├── layout/           # MainLayout (and future AdminLayout)
│   └── pages/            # top-level routed pages (LandingPage, LoginPage, GetStartedPage,
├── HomePage)
│   ├── onboarding/       # 8-step onboarding flow
│   └── engagement/       # profile engagement pages (identity map, avatar, milestones)
└── providers/            # context providers (OnboardingProvider) + redux <Provider>

```



```
wiring
├── services/           # API clients (api.js axios instance + per-resource modules)
├── store/
│   ├── index.js       # configureStore
│   └── slices/         # one slice per domain (auth, profile, engagement, ...)
├── utils/              # classNames, debug, figmaPosition (leaf utilities)
├── tests/
└── e2e/                # Playwright specs
```

3.2 Naming Conventions

Kind	Convention	Example
Components / pages	PascalCase .jsx	Navbar.jsx, HomePage.jsx
Hooks	camelCase, use prefix, .js	useOnboarding.js, useDebounce.js
Utilities	camelCase .js	classNames.js, debug.js
Constants (export name)	UPPER_SNAKE_CASE	STORAGE_KEYS, PROFILE_STAGES
Constants (file)	camelCase	storageKeys.js, profileStages.js
Env vars	VITE_* UPPER_SNAKE	VITE_API_BASE_URL
Slices	<domain>Slice.js	authSlice.js, profileSlice.js
Services	<domain>Service.js (re-exports from api.js)	authService.js, profileService.js
Folders	camelCase or kebab-case for assets	engagement/, getStarted/
Component name (export)	PascalCase, matches file name	export default MainLayout;
Props	camelCase	isActive, onClick, className
Event handlers (internal)	handle + Event	handleSubmit, handleChange
Event handlers (props)	on + Event	onSubmit, onChange
Boolean props	is / has / should prefix	isLoading, hasError, shouldFocus
Test files	tests/e2e/<name>.spec.js	home.spec.js

3.3 Import Order Convention

Apply this order top-to-bottom in every file. Prettier won't reorder for you — discipline matters.

```
// 1. React and framework
import { useEffect, useState } from 'react';
import { useNavigate, useParams } from 'react-router-dom';
import { useDispatch, useSelector } from 'react-redux';
```

```
// 2. Redux slices and actions
import { fetchProfile } from '../store/slices/profileSlice.js';

// 3. Services
import profileService from '../services/profileService.js';

// 4. Custom hooks
import { useOnboarding } from '../hooks/useOnboarding.js';

// 5. Components (shared → ui → cards/sections → page-local)
import Navbar from '../components/shared/Navbar.jsx';
import Button from '../components/ui/Button.jsx';
import EntryMethodCard from '../components/cards/EntryMethodCard.jsx';

// 6. Utilities and constants
import { classNames } from '../utils/classNames.js';
import { debug } from '../utils/debug.js';
import { STORAGE_KEYS } from '../constants/storageKeys.js';

// 7. Assets (icons, images)
import logo from '../assets/brand/gth-mark.png';
```

ESLint's flat config does not auto-sort imports today; if a future change adds `eslint-plugin-import` sort rules, follow that. Until then, the order above is enforced by review.

4. Design System & Theming Reference

4.1 Token Source of Truth

All design tokens live in `src/index.css` inside the `@theme { }` block. Tailwind v4 reads them at build time and emits utilities automatically. **Never hardcode hex values in component files** — if a colour isn't in the theme, add it to the theme first.

The tokens were extracted on **2026-05-04** from Figma file "*Gh Design system - onboarding*" (`Bin8roWL8sloyc36IgFMuT`). See `wiki/design-tokens.md` for the full table and `wiki/figma-node-map.md` for per-token Figma node IDs.

4.2 Colour Families

Every family follows the **same 10-step state ladder** so utilities are predictable.

Suffix	Figma swatch	Use
<code>-light</code>	Light	subtle background fills
<code>-light-hover</code>	Light :hover	
<code>-light-active</code>	Light :active	
<i>(none)</i>	Normal	default fill / button background
<code>-hover</code>	Normal :hover	
<code>-active</code>	Normal :active	
<code>-dark</code>	Dark	strong emphasis fill
<code>-dark-hover</code>	Dark :hover	
<code>-dark-active</code>	Dark :active	
<code>-darker</code>	Darker	deepest emphasis (omitted in <code>success</code>)

Brand families:

- `brand-green` — Ghana-flag green, default action colour. `--color-brand-green` = `#387440` .
- `accent` — Gold accents. `--color-accent` = `#c8951a` .
- `neutral` — Grey-neutrals scale.
- `black` — Foreground / text scale. `bg-black` is `#111111` , not pure `#000` (Tailwind default overridden).
- `yellow` — Warm cream backgrounds (`--color-yellow` = `#fbf4e0`).

Semantic families (Figma `45:11090` *Colors – Semantic colors*):

- `informative` (blue) — informational UI
- `success` (green) — success states. **9 swatches only — no `darker`** .
- `warning` (orange) — warnings
- `danger` (red) — errors, destructive actions

Surface / content / border tokens:

Token	Hex	Use
<code>--color-content-primary</code>	<code>#272e35</code>	primary text
<code>--color-content-secondary</code>	<code>#555f6d</code>	secondary text
<code>--color-content-tertiary</code>	<code>#7e8b99</code>	tertiary text
<code>--color-background-default</code>	<code>#ffffff</code>	default page background
<code>--color-background-subtle</code>	<code>#f5f7f9</code>	subtle section background
<code>--color-border-default</code>	<code>#e9ecef</code>	default border
<code>--color-border-neutral</code>	<code>#555f6d</code>	stronger border

Usage:

```
<button className="bg-brand-green hover:bg-brand-green-hover active:bg-brand-green-active text-white">
  <p className="text-content-primary">
    <div className="bg-background-subtle border border-border-default">
```

4.3 Typography

Two families:

- **Body / paragraphs** — `--font-sans` resolves to `'SF Pro Rounded', -apple-system, BlinkMacSystemFont, 'Nunito', system-ui, sans-serif`. SF Pro Rounded is **Apple's proprietary system font** — Apple devices use it natively; everything else falls back to Nunito (Google Fonts).
- **Display / headings** — `--font-display` resolves to `'Instrument Serif', Georgia, serif`. Loaded from Google Fonts.

The display family auto-applies to any `text-display-*` utility — no need to also write `font-display`.

Display token	Size / line-height	Weight
<code>text-display-xl</code>	72 / 72	600
<code>text-display-lg</code>	52 / 52	600
<code>text-display-md</code>	36 / 40	600
<code>text-display-sm</code>	16 / 19	600
<code>text-display-italic</code>	22 / 32	400 (Instrument Serif)

Paragraph scale (3 weights × 9 sizes, all Inter):

Weight family	Prefix	font-weight	letter-spacing
Strong	<code>text-strong-*</code>	600	0.1 px

Weight family	Prefix	font-weight	letter-spacing
Medium	<code>text-medium-*</code>	450	0.2 px
Normal	<code>text-normal-*</code>	400	0.2 px

Sizes (suffix → font/line in px): `25` (10/16) · `50` (12/20) · `75` (14/24) · `100` (16/28) · `200` (18/32) · `300` (24/36) · `400` (32/44) · `500` (40/56) · `600` (48/64).

Example: `text-strong-100` = Inter SemiBold 16/28, letter-spacing 0.1 px.

4.4 Spacing, Radii, Shadows

- **Spacing** — `--spacing` = `0.25rem` (4 px). Class names map directly to Figma's 8-point grid: `p-1` (4 px), `p-2` (8 px), `p-4` (16 px), `p-6` (24 px), `p-8` (32 px), `p-10` (40 px), `p-12` (48 px), `p-16` (64 px), `p-20` (80 px). Prefer the documented values; skipped numbers (9, 11, 13–15) work mathematically but aren't part of the design vocabulary.
- **Radius** — `rounded-sm` (6 px), `rounded-md` (10 px), `rounded-lg` (16 px), `rounded-xl` (24 px), `rounded-pill` (100 px).
- **Shadows** — `shadow-sm` / `shadow-md` / `shadow-lg` (general), `shadow-bottom-100..400` (standard elevation ladder), `shadow-top-100..400` (above-surface), `shadow-green` (primary CTA glow), `shadow-button-shelf` (4 px "shelf" under primary buttons — Figma `50:8233`).

4.5 Layout Container

`MainLayout` clamps the column to 1728 px and centres it. Inside a section, the standard guide is:

```
<section className="mx-auto w-full max-w-[1728px] px-[clamp(1rem,4vw,2rem)]">
  {/* content */}
</section>
```

Use `clamp()` for fluid horizontal padding rather than breakpoint-based static values (see §19).

5. Component Development Standards

5.1 The Component Pattern

Every UI primitive in `src/components/ui/` follows this shape. Replicate it for new primitives.

```
import { forwardRef } from 'react';
import { classNames } from '../../utils/classNames.js';
import { debug } from '../../utils/debug.js';

const log = debug('Button');

const VARIANTS = {
  primary: 'bg-brand-green border-brand-green-dark text-white',
  secondary: 'bg-accent border-accent-dark text-accent-light',
  tertiary: 'bg-white border-black/30 text-black',
};

const SIZES = {
  lg: 'px-[34px] py-[16px] rounded-[14px]',
  md: 'px-[28px] py-[14px] rounded-[10px]',
  sm: 'px-[18px] py-[12px] rounded-[10px]',
};

const Button = forwardRef(function Button(
  {
    variant = 'primary',
    size = 'lg',
    disabled = false,
    leftIcon,
    rightIcon,
    className = '',
    children,
    ...rest
  },
  ref
) {
  log('render', { variant, size, disabled });

  return (
    <button
      ref={ref}
      type="button"
      disabled={disabled}
      className={classNames(
        'inline-flex items-center justify-center gap-2 transition-all duration-300 ease-in',
        'shadow-button-shelf active:translate-y-1 active:shadow-none',
        VARIANTS[variant],
        SIZES[size],
        disabled && 'opacity-55 cursor-not-allowed',
        className
      )}
      {...rest}
    >
      {leftIcon}
      {children}
      {rightIcon}
    </button>
  );
});
```

```
export default Button;
```

5.2 Mandatory Patterns

1. **forwardRef** — every UI primitive must forward its ref so parents can integrate with form libraries and focus management.
2. **classNames() utility** — never string-concatenate Tailwind classes; never use template literals for conditional classes. `classNames()` lives in `src/utils/classNames.js`.
3. **Variant / size objects** — flat lookup tables, not inline ternaries inside JSX. Makes diffs reviewable and Storybook-style showcases trivial.
4. **...rest spread on the root element** — so consumers can pass `aria-*`, `data-*`, `name`, `id`, etc.
5. **className last** — the consumer-passed `className` must be the last argument to `classNames()` so it overrides defaults.
6. **Controlled + uncontrolled support** — stateful primitives (TextInput, Select, VerificationCode, Checkbox) must accept both `value` (controlled) and `defaultValue` (uncontrolled). Use the `isControlled = value !== undefined` pattern.
7. **State derivation priority** — when multiple prop signals could change visuals, document the precedence ladder. The form primitives use:
`forced state prop → disabled → error → verified → interactive (focus-within)`.
8. **Transitions** — `transition-all duration-300 ease-in` on every interactive element so hover, active, and disabled transitions feel consistent.
9. **Showcase-only state prop** — primitives expose a `state` prop (e.g. `'default' | 'hover' | 'selected' | ...`) so the design-system playground (`/components`) can pin a visual without user interaction. **Production callers omit this** — `:hover` and `:active` drive state naturally.
10. **Single-file components** — one component per file, default export. Compound components (e.g. `Field` + form primitives) can share a folder with a barrel `index.js`.

5.3 Component Categories

Folder	Examples	Purpose
<code>components/ui/</code>	Button, Card, MiniCard, Tag, ProgressBar, ProgressRing, Loader, Modal, Breadcrumbs, Captions, StatusDot, ChatBubble, WatchTutorial, EngagementProgressIndicator	Design-system primitives. Pure, presentational, no app state.
<code>components/ui/form/</code>	Field, TextInput, Textarea, Select, Checkbox, VerificationCode, Upload	Form primitives. Hybrid API (Field props + native <code><input></code> props). Plug into react-hook-form's <code>register()</code> .
<code>components/shared/</code>	Navbar, OnboardingNavbar, OnboardingHeader, Footer, Logo, WavyDivider, modals (Privacy, Terms, Consent)	Cross-feature components. May call into app state.

Folder	Examples	Purpose
<code>components/sections/</code>	<code>landing/HeroSection.jsx</code> , <code>landing/HeroPhotoCard.jsx</code>	Composed page sections (one Figma frame = one section). Pages assemble these.
<code>components/cards/</code>	<code>EntryMethodCard</code> , <code>ProfileStageCard</code> , <code>StageMapNode</code> , <code>MilestoneStatTile</code> , <code>UnlockedFeatureCard</code>	Specialised cards (often feature-specific, often referenced from sections).

5.4 New-Component Checklist

1. Open the Figma frame and capture the node ID (`wiki/figma-node-map.md`).
 2. Enumerate **every visual state** (default, hover, active, focus, disabled, error, selected, loading, empty).
 3. Create the file in the right folder per §3.1.
 4. Implement using the pattern in §5.1.
 5. Add a showcase to `src/pages/HomePage.jsx` covering all variant × size × state permutations.
 6. Append a `wiki/components.md` row with props table, state derivation, and Figma source.
 7. Take a Playwright MCP screenshot at 390 / 768 / 1440 widths and compare to Figma.
 8. Add `log('render', { variant, size, disabled })` at minimum (see §12).
-

6. State Management with Redux Toolkit

6.1 Store

The store lives in `src/store/index.js`. It starts empty — add slices to `reducer` as features land.

```
// src/store/index.js
import { configureStore } from '@reduxjs/toolkit';
import authReducer from '../slices/authSlice.js';
import profileReducer from '../slices/profileSlice.js';
import engagementReducer from '../slices/engagementSlice.js';

export const store = configureStore({
  reducer: {
    auth: authReducer,
    profile: profileReducer,
    engagement: engagementReducer,
  },
});

export default store;
```

When auth ships, wire **redux-persist** here. Pattern (whitelisted to `auth` + `profile`):

```
import { persistStore, persistReducer } from 'redux-persist';
import storage from 'redux-persist/lib/storage';
import { combineReducers } from 'redux';

const persistConfig = { key: 'th-root', storage, whitelist: ['auth', 'profile'] };
const rootReducer = combineReducers({ auth: authReducer, profile: profileReducer, engagement: engagementReducer });

export const store = configureStore({
  reducer: persistReducer(persistConfig, rootReducer),
  middleware: (getDefault) => {
    getDefault({ serializableCheck: { ignoredActions: ['persist/PERSIST', 'persist/REHYDRATE'] } }),
  },
});

export const persistor = persistStore(store);
```

6.2 Slice Template

```
// src/store/slices/profileSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import profileService from '../../services/profileService.js';
import { debug } from '../../utils/debug.js';

const log = debug('profileSlice');

export const fetchProfile = createAsyncThunk(
  'profile/fetch',
  async (_, { rejectWithValue }) => {
    try {
      log('fetchProfile: requesting');
      const { data } = await profileService.getMe();
      log('fetchProfile: ok', { id: data?.id });
      return data;
    } catch (error) {
      return rejectWithValue(error);
    }
  }
);
```

```

    } catch (err) {
      log.error('fetchProfile: failed', {
        status: err.response?.status,
        message: err.message,
      });
      return rejectWithValue(err.response?.data?.message || 'Failed to load profile');
    }
  }
);

const profileSlice = createSlice({
  name: 'profile',
  initialState: {
    me: null,
    isLoading: false,
    error: null,
  },
  reducers: {
    clearError: (state) => { state.error = null; },
    clearProfile: (state) => { state.me = null; },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchProfile.pending, (state) => {
        state.isLoading = true;
        state.error = null;
      })
      .addCase(fetchProfile.fulfilled, (state, action) => {
        state.isLoading = false;
        state.me = action.payload;
      })
      .addCase(fetchProfile.rejected, (state, action) => {
        state.isLoading = false;
        state.error = action.payload;
      });
  },
});

export const { clearError, clearProfile } = profileSlice.actions;
export default profileSlice.reducer;

```

6.3 Custom Hook Wrapper (one per slice)

```

// src/hooks/useProfile.js
import { useDispatch, useSelector } from 'react-redux';
import { fetchProfile, clearError, clearProfile } from '../store/slices/profileSlice.js';

export const useProfile = () => {
  const dispatch = useDispatch();
  const { me, isLoading, error } = useSelector((s) => s.profile);

  return {
    me,
    isLoading,
    error,
    loadProfile: () => dispatch(fetchProfile()),
    clearError: () => dispatch(clearError()),
    clearProfile: () => dispatch(clearProfile()),
  };
};

```

Components never call `useSelector` / `useDispatch` directly — they consume the hook. This keeps slice internals replaceable and selectors centralised.

6.4 Planned Slices

Map each domain to a slice + service + hook (build as features land):

Slice	Service	Hook	Purpose
<code>authSlice</code>	<code>authService</code>	<code>useAuth</code>	Sign up, log in, OTP, refresh, log out, user profile
<code>profileSlice</code>	<code>profileService</code>	<code>useProfile</code>	Talent profile, completeness, persisted across reloads
<code>onboardingSlice</code>	<code>onboardingService</code>	<code>useOnboarding</code>	(Currently a React Context — promote to Redux if cross-route state grows)
<code>engagementSlice</code>	<code>engagementService</code>	<code>useEngagement</code>	XP, milestones, identity map, avatar selections
<code>highlightSlice</code>	<code>highlightService</code>	<code>useHighlight</code>	Highlighting batches (per GTH_Highlighting user stories)
<code>prospectingSlice</code>	<code>prospectingService</code>	<code>useProspecting</code>	Job-prospecting flows (per GTH_JobProspecting)

6.5 When to Use Redux vs Local vs Context

Scope	Choose
Single component, single screen	<code>useState</code>
Multiple components, single screen	Lift state up
Shared across a route subtree (e.g. onboarding flow)	React Context
Shared across the whole app, async, persisted	Redux slice

The onboarding flow uses `OnboardingProvider` (`src/providers/OnboardingProvider.jsx`) because DOB captured on step 01 drives the breadcrumb and the under-18 branch on later steps — that's a route-subtree scope, not whole-app. Don't promote to Redux unless data must outlive the route subtree.

7. API Integration with axios

7.1 The single axios Instance

`src/services/api.js` exports a single configured axios instance. **Every service module imports from this file** — never call `axios.create()` elsewhere.

```
import axios from 'axios';
import { STORAGE_KEYS } from '../constants/storageKeys.js';
import { debug } from '../utils/debug.js';

const log = debug('api');

const baseURL = import.meta.env.VITE_API_BASE_URL || '/api';
log('baseURL:', baseURL);

export const api = axios.create({
  baseURL,
  timeout: 15_000,
  headers: { 'Content-Type': 'application/json' },
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem(STORAGE_KEYS.authToken);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  log('→', config.method?.toUpperCase(), config.url);
  return config;
});

api.interceptors.response.use(
  (response) => {
    log('←', response.status, response.config.url);
    return response;
  },
  (error) => {
    log.error('x', error.response?.status, error.config?.url, error.message);
    return Promise.reject(error);
  }
);

export default api;
```

7.2 Refresh-Token Flow (to wire when auth ships)

Add a 401-retry interceptor that:

1. Catches 401 on a non-refresh endpoint.
2. POSTs to the refresh endpoint with `STORAGE_KEYS.refreshToken`.
3. On success, updates `STORAGE_KEYS.authToken` and replays the original request.
4. On failure, clears all storage keys and dispatches a logout action that redirects to `/login`.

Track the implementation against `wiki/api.md` (Refresh-Token Flow section) and the relevant user stories in `docs/user stories/`.

7.3 Service Module Pattern

```
// src/services/profileService.js
import { api } from './api.js';

const profileService = {
  getMe: () => api.get('/profile/me'),
  updateMe: (data) => api.patch('/profile/me', data),
  uploadAvatar: (formData) => api.post('/profile/avatar', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  }),
};

export default profileService;
```

7.4 Planned Services

Document as you add — keep this table mirrored in `wiki/api.md`.

Service	Endpoint	Key methods
<code>authService</code>	<code>/auth</code>	<code>signUp</code> , <code>login</code> , <code>verify0tp</code> , <code>resend0tp</code> , <code>refresh</code> , <code>logout</code>
<code>profileService</code>	<code>/profile</code>	<code>getMe</code> , <code>updateMe</code> , <code>uploadAvatar</code>
<code>onboardingService</code>	<code>/onboarding</code>	<code>saveStep</code> , <code>submitFinal</code> , <code>getDraft</code>
<code>engagementService</code>	<code>/engagement</code>	<code>getMilestones</code> , <code>claimMilestone</code> , <code>saveAvatar</code>
<code>highlightService</code>	<code>/highlights</code>	<code>list</code> , <code>submit</code> , <code>attachMedia</code>
<code>prospectingService</code>	<code>/prospecting</code>	<code>getMatches</code> , <code>applyTo0ppportunity</code> , <code>bookmark</code>

7.5 Calling axios from Components

Never call `api.get()` directly from a component. Always go through a service module imported by a thunk, then expose state via a hook (see §6.3). The two-layer indirection makes mocking trivial for Playwright and keeps endpoint shape change-resilient.

8. Forms with react-hook-form + zod

react-hook-form orchestrates form state; zod validates. `@hookform/resolvers/zod` is the bridge.

8.1 Standard Form Pattern

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { TextInput } from '../components/ui/form';
import Button from '../components/ui/Button.jsx';
import { debug } from '../utils/debug.js';

const log = debug('LoginForm');

const schema = z.object({
  email: z.string().email('Enter a valid email'),
  password: z.string().min(8, 'At least 8 characters'),
});

const LoginForm = ({ onSubmit }) => {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm({
    resolver: zodResolver(schema),
    mode: 'onBlur',
  });

  const submit = async (values) => {
    log('submit:', { email: values.email });
    await onSubmit(values);
  };

  return (
    <form onSubmit={handleSubmit(submit)} className="flex flex-col gap-4">
      <TextInput
        label="Email"
        required
        type="email"
        error={errors.email?.message}
        {...register('email')}
      />
      <TextInput
        label="Password"
        required
        type="password"
        error={errors.password?.message}
        {...register('password')}
      />
      <Button type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Signing in...' : 'Sign in'}
      </Button>
    </form>
  );
};
```

8.2 Form Conventions

- **Schema lives next to the form** (or under `src/schemas/` for cross-page reuse). Always use zod — never roll a custom validator.
- **Use the `form/` primitives** — they expose Field props (`label` , `required` , `helperText` , `error` , `verified`) plus native `<input>` props so `register()` integrates cleanly. See `wiki/components.md` for state derivation per primitive.
- **mode: `'onBlur'`** for most forms — fast feedback without thrashing on every keystroke. Use `'onChange'` for guided flows (OTP, password rules) where instant feedback is the UX intent.
- **Error display** — pass `error={errors.<field>?.message}` to the primitive; the primitive applies `aria-invalid` + `role="alert"` automatically.
- **Disable on submit** — gate the submit button on `isSubmitting` and show a `<Loader size="sm" />` inline.
- **No silent failures** — always surface server-side errors. For form-wide errors, set `setError('root', { message })` and render at the top of the form.

8.3 Onboarding Flow Specifics

The 8-step onboarding flow shares state across steps via `OnboardingProvider`. Each step page:

1. Reads its slice of state via `useOnboarding()`.
2. Renders the appropriate form primitives.
3. On submit, validates with zod, writes to the provider, and navigates to the next step.
4. The final review page submits the aggregated payload via `onboardingService.submitFinal()`.

DOB on step 01 drives downstream branching (Parent step appears for under-18 talents). Encode that branch in the route logic, not by hiding the step at render time — visibility-only solutions break breadcrumb math.

9. Routing & Layout Architecture

9.1 Routes Today

Config-based routing in `src/App.jsx`. **Not file-based**, no Next.js conventions.

Path	Element	Layout
<code>/</code>	<code>LandingPage</code>	<code>MainLayout</code>
<code>/login</code>	<code>LoginPage</code>	<code>MainLayout</code> (onboarding chrome)
<code>/get-started</code>	<code>GetStartedPage</code>	<code>MainLayout</code> (onboarding chrome)
<code>/onboarding/talent/welcome</code>	<code>OnboardingWelcomePage</code>	<code>MainLayout</code> + <code>OnboardingProvider</code>
<code>/onboarding/talent/dob</code>	<code>OnboardingDobPage</code>	same
<code>/onboarding/talent/personal-info</code>	<code>OnboardingPersonalInfoPage</code>	same
<code>/onboarding/talent/contact</code>	<code>OnboardingContactPage</code>	same
<code>/onboarding/talent/address</code>	<code>OnboardingAddressPage</code>	same
<code>/onboarding/talent/education</code>	<code>OnboardingEducationPage</code>	same
<code>/onboarding/talent/parent-info</code>	<code>OnboardingParentInfoPage</code>	same
<code>/onboarding/talent/review</code>	<code>OnboardingReviewPage</code>	same
<code>/components</code>	<code>HomePage</code> (DS playground)	<code>MainLayout</code>
<code>/profile/engagement</code>	<code>ProfileEngagementPage</code>	own chrome
<code>/profile/engagement/identity</code>	<code>IdentityMapPage</code>	own chrome
<code>/profile/engagement/milestone</code>	<code>MilestoneUnlockPage</code>	own chrome
<code>/profile/engagement/milestone/top-20</code>	<code>Top20MilestonePage</code>	own chrome
<code>/profile/engagement/avatar</code>	<code>AvatarCustomiserPage</code>	own chrome
<code>/profile/engagement/avatar/{skin, hair, extras, outfit}</code>	<code>Avatar*Page</code>	own chrome

9.2 Reserved Routes

Path	Plan
<code>/admin/*</code>	Lazy-loaded admin subsystem (<code>const AdminApp = lazy(() => import('./admin/AdminApp'))</code>) wrapped in <code><Suspense fallback={<Loader />>}</code> . Mirrors elysium's pattern.

9.3 MainLayout & Chrome Switching

`src/layout/MainLayout.jsx` renders different chrome based on `pathname`. Both the navbar **and the footer** switch — onboarding paths get the slim variants, everything else gets the full marketing chrome:

```
const ONBOARDING_CHROME_PATHS = ['/onboarding/', '/login', '/get-started'];

const MainLayout = () => {
  const { pathname } = useLocation();
  const onboarding = ONBOARDING_CHROME_PATHS.some(
    (p) => pathname === p || pathname.startsWith(p.endsWith('/') ? p : `${p}/`)
  );

  return (
    <div className="min-h-screen mx-auto flex w-full max-w-[1728px] flex-col">
      {onboarding ? <OnboardingNavbar /> : <Navbar />}
      <main className="flex-1"><Outlet /></main>
      {onboarding ? <OnboardingFooter /> : <Footer />}
    </div>
  );
};
```

⚠ The current code renders `null` instead of `<OnboardingFooter />` on onboarding paths — that's an outstanding bug against the Figma spec (see §1.2). Add `src/components/shared/OnboardingFooter.jsx` (slim copyright + legal-link row), wire it here, and update `wiki/components.md` + `wiki/routing.md` + `wiki/log.md`.

If you add a route that should use onboarding chrome, append its prefix to `ONBOARDING_CHROME_PATHS`. Update `wiki/routing.md` and `wiki/log.md`.

9.4 Protected Routes (to wire when auth ships)

Pattern (place under `src/components/shared/` or `src/routes/`):

```
// src/components/shared/ProtectedRoute.jsx
import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

const ProtectedRoute = ({ allow = 'authenticated' }) => {
  const { isAuthenticated, user } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }
  if (allow === 'admin' && user?.role !== 'admin') {
    return <Navigate to="/" replace />;
  }
  return <Outlet />;
};

export default ProtectedRoute;
```

9.5 Adding a Route — Steps

1. Create the page file in `src/pages/<Name>Page.jsx` (or under a sub-folder for feature groupings).
 2. (Optional) Add the path to `src/constants/routes.js` once that constants file is introduced.
 3. Register the route in `src/App.jsx` under the right layout.
 4. Add a nav-link in `Navbar.jsx` or `OnboardingNavbar.jsx` if user-facing.
 5. Add a Playwright spec under `tests/e2e/` for non-trivial pages (one `.spec.js` per route minimum).
 6. Update `wiki/routing.md` and `wiki/components.md` ; append an entry to `wiki/log.md` .
-

10. Page Development Procedures

10.1 Page Structure Pattern

Pages compose **sections**. Page files contain almost no logic — they import section components and arrange them.

```
// src/pages/LandingPage.jsx
import { debug } from '../utils/debug.js';
import HeroSection from '../components/sections/landing/HeroSection.jsx';
import HeroPhotoCard from '../components/sections/landing/HeroPhotoCard.jsx';

const log = debug('LandingPage');

const LandingPage = () => {
  log('mount');
  return (
    <>
      <HeroSection />
      { /* Future: <FeaturesSection /> <TestimonialSection /> <CTASection /> etc. */ }
    </>
  );
};

export default LandingPage;
```

If a page needs data, it does so via slice hooks (§6.3) — never directly through axios.

10.2 Pages Inventory & Owners

Map each page to its source-of-truth user-story doc and Figma node. Keep this synced with [wiki/components.md](#).

Page	User-story doc	Notes
LandingPage	(marketing)	Hero + dark-green bg + photo card composition + trust marquee
LoginPage	GTH_Onboarding_v1.3_POST_REVIEW_1.docx	Aside imagery + email/password form. Slim onboarding chrome.
GetStartedPage	GTH_Onboarding_v1.3_POST_REVIEW_1.docx	Role select (Card components in radiogroup pattern).
OnboardingWelcomePage	GTH_Onboarding_v1.3_POST_REVIEW_1.docx	Intro / consent.
OnboardingDobPage	same	DOB capture — drives under-18 branch.
OnboardingPersonalInfoPage	same	Name + identity fields.
OnboardingContactPage	same	Email, phone, OTP verification.
OnboardingAddressPage	same	Region + town.
OnboardingEducationPage	same	Level + grade (cascading Selects).

Page	User-story doc	Notes
OnboardingParentInfoPage	same	Conditional — only renders for under-18 talents.
OnboardingReviewPage	same	Aggregated payload review + submit.
HomePage	(internal — DS playground)	Renders all primitives × all states for visual QA.
ProfileEngagementPage	GTH_Engagement_Batch1_v1_0.docx	Stage card + identity map entry + milestone strip.
IdentityMapPage	GTH_Engagement_Batch1_v1_0.docx	Identity map mosaic + entry method cards.
MilestoneUnlockPage	GTH_Engagement_Batch2_v1_0.docx	Stat tiles + unlocked features.
Top20MilestonePage	GTH_Engagement_Batch2_v1_0.docx	Top-20 milestone celebration.
AvatarCustomiserPage (+ skin/hair/extras/outfit)	GTH_Engagement_Batch3_v1_1.docx	Avatar customiser with hero stage + customiser panel.

10.3 Building a Page — End-to-End

1. **Read the user-story doc** in `docs/user stories/`. Note the acceptance criteria and edge cases.
2. **Pull design specs** from Figma via Figma MCP `get_design_context` / `get_metadata` (see §19). Save outputs to a temp scratch file if the frame is large.
3. **List the sections** the page comprises and what state each needs (local, context, or Redux per §6.5).
4. **Build the sections** in `src/components/sections/<page>/`. Add `debug(scope)` logs (§12).
5. **Compose the page** under `src/pages/`.
6. **Add the route** (§9.5).
7. **Verify:**
 - `npm run lint` clean
 - `npm run format:check` clean
 - `npm run build` clean
 - Playwright spec passes for the route
 - Playwright MCP screenshot at 390 / 768 / 1440 matches Figma
8. **Update the wiki** — append component rows, route rows, and a `log.md` entry.
9. **Commit + push** following §15.

11. Wiki Discipline (the source of truth)

11.1 Why a Wiki

This repo follows the [Karpathy LLM-Wiki pattern](#): a **living, persistent, compounding artifact** that captures verified knowledge so future sessions (human or AI) start from a higher floor. **Read `wiki/INDEX.md` at the start of every non-trivial change.**

11.2 Confidence Tags (mandatory on every claim)

Tag	Meaning
<code>☐ VERIFIED</code>	Confirmed by reading current source code or authoritative docs
<code>☐ LIKELY</code>	Inferred from code patterns or adjacent evidence, not directly confirmed
<code>☐ NEEDS-CLARIFICATION</code>	Ambiguous — needs human input
<code>⚠ ASSUMPTION</code>	Educated guess, may be wrong
<code>☐ STALE</code>	Was true at one point, may no longer be accurate
<code>☐ EXTERNAL</code>	Depends on external system (API, Figma, etc.) — verify before acting

Trust but verify. If a wiki claim has any tag other than `☐ VERIFIED`, re-check it against current source before acting on it. After verification, upgrade the tag.

11.3 When to Update the Wiki

After any significant change:

- Append an entry to `wiki/log.md` in the format `## [YYYY-MM-DD] action | subject` followed by a short paragraph of context.
- Update the affected wiki page(s).
- Keep confidence tags accurate. A change that broke a `☐ VERIFIED` claim demotes it to `☐ STALE` until re-verified.

11.4 Wiki Pages

Page	Owns
<code>INDEX.md</code>	Table of contents + confidence-tag legend
<code>architecture.md</code>	Stack, directory tree, entry points
<code>routing.md</code>	Route map, layouts, future auth guards
<code>api.md</code>	axios client, interceptors, env vars, planned endpoints
<code>components.md</code>	Component catalog (props, state, Figma source)
<code>design-tokens.md</code>	Tailwind v4 tokens (colors, type, spacing, radii)
<code>figma-fidelity.md</code>	Pixel-perfect rules + <code>clamp()</code> + MCP workflow

Page	Owns
<code>figma-node-map.md</code>	Figma frame → node-ID lookup
<code>debugging-workflow.md</code>	Debug-log discipline + Playwright verification
<code>pre-commit-workflow.md</code>	Husky, lint-staged, commitlint, lint fixes
<code>log.md</code>	Chronological record of changes

12. Debug-Log Discipline (mandatory)

Adapted from the MangoTV / sibling-project convention. Full details in [wiki/debugging-workflow.md](#).

12.1 The `debug` Helper

Use `debug(scope)` from `src/utils/debug.js`. It auto-prefixes lines and **only fires in dev** — production bundles stay silent because `import.meta.env.DEV` is `false`.

```
import { debug } from '../utils/debug.js';
const log = debug('HomePage');

log('mount');
log('fetching content...');
log('categories loaded:', categories.length);
log.warn('unexpected branch');
log.error('failed to load:', { error: err.message, endpoint });
```

If you absolutely must use raw `console`, mirror the format: `console.log('[ComponentName] step:', data)`.

12.2 Where Logs are Required

1. **Async operations** — log before, log response shape (not full payload), log errors with context (id, type, endpoint).
2. **Component mount** — log key props/state.
3. **Conditional branches** — log which branch was taken and why.
4. **Redux dispatch** — log action type + key payload fields.
5. **API response** — log status + URL + data shape.
6. **Style-affecting calculations** — log computed values (widths, item counts, derived states).

12.3 Verifying with Playwright MCP

After writing code:

1. Start the dev server (`npm run dev`) or rely on Playwright's `webServer` auto-boot.
2. Use Playwright MCP `browser_navigate` to the affected page.
3. Use `browser_console_messages` to verify logs appear in the expected order with expected payloads.
4. Use `browser_take_screenshot` for visual verification.
5. Compare against Figma if a design exists.

Browser console errors are real bugs — never treat them as background noise.

12.4 Log Lifecycle

- **Keep logs during active development.** They're the audit trail when something breaks.

- **Cleanup pass before release** strips redundant logs in one sweep — not piecemeal during the build-out.
 - Production builds are silent (no flag-flip needed), so logs that survive cleanup are zero-cost in prod.
-

13. AI-Augmented Workflow (Claude Code + MCP)

13.1 The Development Cycle

Follow this end-to-end for any non-trivial change.

Step 1 – RECEIVE TASK

- Read the user-story doc in docs/user stories/.
- Read the linked Figma frame.
- Identify acceptance criteria & edge cases.

Step 2 – READ THE WIKI

- wiki/INDEX.md, then the relevant page(s).
- Note any non-VERIFIED claims that could derail you – re-verify if so.

Step 3 – BRAINSTORM (with Claude)

- Describe the task, paste the user story, link the Figma node.
- Ask Claude for an approach. Push back; don't accept the first idea.
- Decide: where does state live? What primitives are reused? What's new?

Step 4 – WRITE THE PLAN

- Multi-step work needs a written plan (phases, files touched, tests).
- For greenfield components, plan the showcase entries too.

Step 5 – TDD WHERE PRACTICAL

- Write Playwright specs / unit tests first for behaviour you can express.
- Don't skip because "the change is small." Smell-test: if you'd ship this without testing it, the test was probably the most important part.

Step 6 – IMPLEMENT IN PHASES

1. File scaffolding (folders, exports, route wiring).
 2. Markup + Tailwind styling.
 3. State + thunks + service calls.
 4. Interactive states + transitions + event handlers.
 5. Edge cases – loading, error, empty, disabled, error-recovery.
- Add debug() logs throughout.

Step 7 – VERIFY (Verification-Before-Completion)

Run and quote the output of:

- npm run lint
- npm run format:check
- npm run build
- npm run test:e2e (or the targeted spec)
- Playwright MCP browser_navigate + browser_console_messages + browser_take_screenshot

Evidence before assertions. Don't claim "done" without quoting at least lint + build + a screenshot.

Step 8 – DEBUG SYSTEMATICALLY (if anything failed in step 7)

- Find the root cause, not a symptom patch.
- Add more debug() logs. Compare actual vs expected.
- Read the relevant slice / interceptor / wiki page before guessing.

Step 9 – UPDATE THE WIKI

- Update components.md / routing.md / api.md / design-tokens.md as needed.
- Append a log.md entry.

Step 10 – COMMIT & PUSH

- Small, logical commits. Conventional-commits format.

- Husky + commitlint will block if the message or staged files don't pass.

Step 11 – CODE REVIEW

- Request review for major features (see §18).
- For trivial changes, self-review against the §18 checklist before merge.

13.2 Prompt Template — Implementation Plan

Copy + fill in:

```
I'm working on the TalentHub frontend (Vite 7, React 19, Tailwind v4, React Router 6,
Redux Toolkit, axios, react-hook-form + zod, Playwright).

TASK: [e.g. "Build the OnboardingEducationPage with cascading Level-Grade selects"]

WIKI CONTEXT (already read):
- wiki/architecture.md
- wiki/routing.md
- wiki/components.md (Select details, Field details)
- wiki/design-tokens.md
- wiki/figma-fidelity.md
USER STORY: docs/user stories/GTH_Onboarding_v1.3_POST_REVIEW_1.docx
FIGMA NODES: [e.g. 2849:66712, ...] (verified via get_design_context)

PROJECT CONVENTIONS:
- forwardRef + classNames() + variant objects + ...props spread
- Tailwind v4 tokens (bg-brand-green, text-display-md, rounded-pill, ...)
- clamp() for all scalable spacing/sizing
- debug(scope) logs at every async/branch/dispatch/calc
- react-hook-form + zod schemas
- Service modules → thunks → custom hook (no direct useSelector/useDispatch in components)

FILES TO CREATE/MODIFY:
- src/pages/onboarding/OnboardingEducationPage.jsx
- src/schemas/educationSchema.js
- (maybe) extension to OnboardingProvider for education state

REQUIREMENTS:
- [acceptance criteria from user story]
- Handle loading / error / empty states
- Cascading Select pattern (Grade disabled until Level chosen)
- Under-18 branch propagation to ParentInfo step on submit

Please give me a phased plan with code for each phase. After the plan,
list the verification commands you'd run.
```

13.3 Prompt Template — Debugging

```
I'm seeing [symptom] on [page or component].

EXPECTED: [what should happen]
ACTUAL: [what's happening]
CONSOLE OUTPUT:
[paste relevant log lines from browser_console_messages]
NETWORK:
[relevant requests + statuses]

What I've already tried:
- [thing 1]
```

- [thing 2]

Please diagnose the root cause and propose a fix. Add debug() logs if needed to isolate the data flow.

13.4 MCP Tools Cheat Sheet

MCP	Purpose	Notable tools
Playwright MCP	Drive a real browser for visual + console verification	<code>browser_navigate</code> , <code>browser_console_messages</code> , <code>browser_take_screenshot</code> , <code>browser_resize</code>
Figma MCP (HTTP)	Read design specs from the Figma file (must be started via Figma desktop's Dev Mode MCP)	<code>get_design_context</code> , <code>get_metadata</code> , <code>get_variable_defs</code> , <code>get_screenshot</code>

Start Figma MCP under *Figma → Preferences → Enable Dev Mode MCP Server* before any task that needs design extraction. Restart Claude Code whenever you edit `.mcp.json`.

13.5 What to Delegate vs Drive Yourself

Delegate to AI	Drive yourself
Boilerplate scaffolding, repetitive variant tables	Architecture decisions (where does state live?)
Migration mechanics, refactors, rename passes	API contracts, security boundaries
Test scaffolding, fixture creation	Final review of generated code (always)
Pulling Figma specs into Tailwind classes via MCP	Anything touching auth / payment / personal data flows
Cleanup passes (debug-log stripping, unused imports)	Decisions about adding new dependencies

14. Testing & Verification

14.1 Playwright E2E (primary safety net)

`playwright.config.js` boots the dev server on `:5173` and runs Chromium-only by default. Specs live under `tests/e2e/`.

Add a spec for every new route at minimum:

```
// tests/e2e/login.spec.js
import { test, expect } from '@playwright/test';

test.describe('LoginPage', () => {
  test('renders the email + password form', async ({ page }) => {
    await page.goto('/login');
    await expect(page.getByLabel('Email')).toBeVisible();
    await expect(page.getByLabel('Password')).toBeVisible();
    await expect(page.getByRole('button', { name: /sign in/i })).toBeVisible();
  });

  test('shows validation errors on empty submit', async ({ page }) => {
    await page.goto('/login');
    await page.getByRole('button', { name: /sign in/i }).click();
    await expect(page.getByText(/enter a valid email/i)).toBeVisible();
    await expect(page.getByText(/at least 8 characters/i)).toBeVisible();
  });
});
```

Run modes:

- `npm run test:e2e` — headless, full suite (CI-compatible)
- `npm run test:e2e:headed` — visible browser
- `npm run test:e2e:ui` — Playwright UI mode (best for authoring + debugging)

The dev server is reused across tests when `process.env.CI` is unset (see `playwright.config.js`).

14.2 Per-Component Manual QA Checklist

Before pushing any UI change:

- ☐ All variant styles render correctly in the design-system playground (`/components`).
- ☐ Hover, active, focus, and disabled states behave per Figma.
- ☐ Component scales correctly at 390 / 768 / 1024 / 1440 widths (`browser_resize` via Playwright MCP).
- ☐ No errors or warnings in the browser console (`browser_console_messages`).
- ☐ Fonts and colors match design tokens (DevTools → Computed → font-family + colors should be CSS variable references).
- ☐ Keyboard navigation works (Tab to focus, Enter/Space to activate).

- ☐ ARIA attributes present where required (see §17).
- ☐ `debug()` logs fire in the expected order (`browser_console_messages` shows the breadcrumbs).

14.3 Common Issues Reference

Symptom	Likely cause	Fix
Component not rendering	Wrong import path or wrong export	Check casing on the file path; check default vs named export
Styles not applying	Token missing from <code>@theme</code>	Add the token to <code>src/index.css</code> ; restart dev server
Hover effect missing	Missing <code>hover:</code> prefix	Add the variant explicitly (<code>hover:bg-brand-green-hover</code>)
Redux state is <code>undefined</code>	Slice not added to <code>store.reducer</code>	Wire the reducer in <code>src/store/index.js</code>
API returns 401	Token missing or expired	Inspect <code>localStorage[STORAGE_KEYS.authToken]</code> ; verify interceptor attaches
Infinite re-render loop	<code>useEffect</code> missing deps / new ref each render	Stabilise refs with <code>useMemo</code> / <code>useCallback</code> ; provide a correct dep array
<code>VITE_API_BASE_URL</code> undefined	<code>.env</code> missing or dev server not restarted	Confirm <code>.env</code> exists; restart <code>npm run dev</code>
Blank page after route change	Missing <code><Outlet /></code> in layout	Ensure <code>MainLayout</code> renders <code><main><Outlet /></main></code>
Playwright <code>webServer</code> never starts	Port 5173 in use	Kill the existing process or change the port in <code>playwright.config.js</code>
Husky hooks not running	<code>.husky/_</code> directory missing	Re-run <code>npm install</code> (Husky's <code>prepare</code> script re-installs hooks)

15. Git Workflow & Pre-Commit Hooks

15.1 Branch Strategy

```
main (protected – production-ready)
├─ develop (integration branch)
│   ├── feat/landing-hero
│   ├── feat/onboarding-education
│   ├── feat/avatar-customiser
│   ├── feat/auth-flow
│   ├── fix/onboarding-dob-validation
│   └─ chore/wiki-tokens-refresh
```

Rules:

1. Never push directly to `main` or `develop`. Always branch.
2. Branch naming: `feat/<short-name>`, `fix/<short-name>`, `chore/<short-name>`, `docs/<short-name>`, `refactor/<short-name>`.
3. Pull from `develop` before starting (`git pull origin develop --rebase` or merge — pick one and stick to it).
4. Small, frequent pushes. One logical change per commit.
5. Always open a PR for review before merging to `develop`.

15.2 Pre-Commit (Husky + lint-staged)

`.husky/pre-commit` runs `npm run lint-staged`, which is configured in `package.json` to run:

File pattern	Action
<code>*.{js,jsx}</code>	<code>eslint --fix</code> then <code>prettier --write</code>
<code>*.{json,css,md}</code>	<code>prettier --write</code>

15.3 Commit-Msg (commitlint)

`.husky/commit-msg` runs `npm run commitlint --edit "$1"`. Commits must match conventional-commits format:

```
type(scope): subject
[optional body]
[optional footer]
```

Type	When
<code>feat</code>	new feature or component
<code>fix</code>	bug fix
<code>chore</code>	config, deps, tooling

Type	When
docs	documentation only
refactor	code restructuring (no behaviour change)
test	adding / fixing tests
style	CSS / styling only
perf	performance improvement
ci	CI config
build	build pipeline
revert	reverting a prior commit

Header limit is **100 chars** (raised from the default 72 in `commitlint.config.js`).

Examples:

```
feat(onboarding): add cascading Level → Grade selects on education step
fix(api): retry on 401 with refresh token before clearing storage
chore(deps): bump vite to 7.1.0
docs(wiki): record design-token refresh from Figma 2026-05-04
refactor(layout): extract onboarding-chrome predicate to its own helper
test(landing): cover hero CTA scroll behaviour at 1440 width
```

15.4 Before Pushing — Mandatory Verification

Run all four clean:

```
npm run lint
npm run format:check
npm run build
npm run test:e2e
```

If anything fails, **fix the underlying issue**. Don't bypass with `--no-verify`. Quote the passing output in the PR description so reviewers see the evidence.

15.5 Recovering From a Bad Commit

```
# See the last 10 commits
git log --oneline -10

# Option A – undo last commit, keep changes staged
git reset --soft HEAD~1

# Option B – undo last commit AND discard changes (destructive)
git reset --hard HEAD~1

# Option C – create a new "undo" commit for an arbitrary commit (safest on shared branches)
git revert <commit-hash>

# Option D – restore a single file from an earlier commit
git checkout <commit-hash> -- src/components/ui/Button.jsx
```

After a Husky hook failure, **don't** `git commit --amend` — make a new commit instead. Amending after a hook failure can leave the index in a confusing state.

16. Security & Performance Standards

16.1 Security Checklist

- **No hardcoded secrets.** Every secret lives in `.env` with a `VITE_` prefix (and only if it's safe to ship to the client). Server-only secrets never go into `VITE_*`.
- **Auth tokens** live in `localStorage` today (via `STORAGE_KEYS.authToken`). When the backend supports `httpOnly` cookies, migrate via the universal-cookie helper that's already a dependency. Document the migration in `wiki/api.md`.
- **Never use** `dangerouslySetInnerHTML` with user-generated content. If you must render HTML, sanitise via a vetted library before render.
- **Validate every form input** client-side via a zod schema (§8). Server validation is non-negotiable too — never trust the client alone.
- **HTTPS only** for production API endpoints. The axios instance honours `VITE_API_BASE_URL` — set it to an HTTPS URL in production builds.
- **No sensitive data in logs.** The `debug()` helper auto-silences in prod, but you must still avoid logging passwords, tokens, OTPs, or full PII payloads even in dev. Log shapes (`{ status, id }`), not values.
- **No** `eval()`, `new Function()`, **or dynamic-string** `import()`. ESLint won't catch all of these — discipline matters.
- **Dependencies:** review every new package against its size, maintenance, and trust footprint before adding. `npm audit` after every install; resolve `high` / `critical` findings before pushing.

16.2 Performance Checklist

- **Code splitting** — lazy-load heavy or admin-only subtrees:

```
import { lazy, Suspense } from 'react';
const AdminApp = lazy(() => import('./admin/AdminApp.jsx'));
<Suspense fallback={<Loader size="lg" />}>
  <AdminApp />
</Suspense>
```

Vendor / router / redux are already chunked separately in `vite.config.js`.

- **Memoise expensive renders** — `React.memo` for pure presentational components rendered in lists; `useMemo` for computed/filtered arrays; `useCallback` for handlers passed to memoised children.
- **Avoid inline objects/arrays in JSX:**

```
// BAD – new object every render
<Card style={{ padding: 10 }} />

// GOOD – stable reference
const cardStyle = useMemo(() => ({ padding: 10 }), []);
<Card style={cardStyle} />
```

- **Debounce inputs** that trigger searches or API calls (`useDebounce` lives in `src/hooks/` — add it when needed).
 - **Paginate** all list endpoints. Don't fetch full collections.
 - **Images**: prefer WebP / AVIF, set explicit `width` + `height` or `aspect-ratio` to prevent layout shift. Use `loading="lazy"` for below-the-fold images.
 - **Fonts**: preconnect to Google Fonts in `index.html` ; subset SF Pro Rounded fallback to avoid layout shift between Apple and non-Apple.
 - **Tailwind purge** is automatic in v4 via the Vite plugin — don't manually configure content paths unless you add file types Vite doesn't already index.
-

17. Accessibility Standards

WCAG 2.1 AA is the floor, not the ceiling.

- **Semantic HTML first.** `<nav>`, `<main>`, `<section>`, `<article>`, `<button>`, `<form>`, `<ol>` / `<ul>` / `<li>`. Don't render an interactive element as a `<div>` with `onClick`.
 - **Every `<img>` has meaningful `alt` text.** Use `alt=""` for purely decorative images.
 - **Icon-only buttons get `aria-label`**. Same for icon-only links.
 - **Form fields have associated labels.** The `Field` primitive does this for you via `htmlFor` + auto-generated `id`. If you compose `Field` manually, supply both.
 - **Errors get `role="alert"`** (the form primitives already do).
 - **Focus management** — visible focus rings are mandatory. Tailwind's default focus utilities are fine; the form primitives apply `focus-within` styles automatically.
 - **Keyboard support** — every interactive element is focusable and activatable with Enter or Space. Use the `Tag` of `<button>` for actions and `<a>` for navigation; don't swap them.
 - **Live regions** — character counters, OTP completion announcements, etc. use `aria-live="polite"` (Textarea does this).
 - **Colour contrast** — the documented palette meets AA when used as intended (foreground tokens on background tokens). Avoid `text-content-tertiary` on light yellow surfaces — verify.
 - **Reduced motion** — respect `prefers-reduced-motion`. Wrap entrance animations and large parallax effects in a media query.
-

18. Code Review Framework

18.1 How to Conduct a Review

1. `git fetch && git checkout <branch>` and pull.
2. `npm install` if dependencies changed.
3. Run **every verification command** the author should have run:
 - `npm run lint`
 - `npm run format:check`
 - `npm run build`
 - `npm run test:e2e`
4. `npm run dev` and visually inspect the affected pages in a browser at 390 / 768 / 1440 widths. Optionally drive via Playwright MCP for a recorded walkthrough.
5. Walk through the checklist below. Mark each item **P** (pass), **F** (fail), or **N/A**.
6. For each **F**, write a review comment in the format in §18.3.
7. If all **P**, approve. Otherwise, request changes.

18.2 Review Checklist

A. File structure & naming

#	Check	P/F/NA
A1	File is in the correct folder per §3.1	
A2	File name follows the convention per §3.2	
A3	Import order follows §3.3	
A4	No unused imports	
A5	Component / export name matches file name	

B. Component patterns

#	Check	P/F/NA
B1	UI primitive uses <code>forwardRef</code>	
B2	Uses <code>classNames()</code> (no manual string concatenation)	
B3	Variants / sizes are flat lookup tables, not nested ternaries	
B4	<code>...rest</code> spread on the root element	
B5	Consumer <code>className</code> is the LAST argument to <code>classNames()</code>	
B6	Sensible default values for every prop	
B7	Boolean props use <code>is</code> / <code>has</code> / <code>should</code> prefix	
B8	Event-handler props use <code>on</code> prefix; internal handlers use <code>handle</code>	

#	Check	P/F/NA
B9	Stateful primitive supports both controlled + uncontrolled	

C. Tailwind & design system

#	Check	P/F/NA
C1	Uses design-token colours (no raw <code>#387440</code> etc.)	
C2	Uses design-token typography (<code>text-display-md</code> , <code>text-medium-100</code>)	
C3	Hover / active / disabled implemented with <code>hover:</code> / <code>active:</code> / <code>disabled:</code>	
C4	Transitions use <code>transition-all duration-300 ease-in</code>	
C5	No inline styles except CSS-variable references or computed <code>clamp()</code>	
C6	Scalable properties use <code>clamp()</code> (not breakpoint-based static px)	
C7	New tokens added to <code>@theme</code> , not hardcoded	

D. Redux & state

#	Check	P/F/NA
D1	Slice follows the template (initial state with isLoading / error)	
D2	Async thunks use <code>createAsyncThunk</code> with <code>rejectWithValue</code>	
D3	Component consumes a custom hook, not <code>useSelector</code> / <code>useDispatch</code> directly	
D4	Loading state renders a loader / skeleton	
D5	Error state renders an error message	
D6	Empty state is handled	

E. API integration

#	Check	P/F/NA
E1	Uses the centralised <code>api.js</code> instance	
E2	No hardcoded API URLs (<code>import.meta.env.VITE_API_BASE_URL</code>)	
E3	Error handling present in thunks (try/catch + <code>rejectWithValue</code>)	
E4	Service is one module per resource, default-exported	
E5	Parameters sanitised / validated before send	

F. Forms

#	Check	P/F/NA
F1	zod schema present and exhaustive	
F2	<code>react-hook-form</code> with <code>zodResolver</code>	

#	Check	P/F/NA
F3	Submit button disabled on <code>isSubmitting</code>	
F4	Errors surface via the form primitives' <code>error</code> prop	
F5	Server-side errors mapped to <code>setError('root', ...)</code> or per-field	

G. Security & performance

#	Check	P/F/NA
G1	No hardcoded secrets / tokens / keys	
G2	No <code>dangerouslySetInnerHTML</code> with user content	
G3	No sensitive data in <code>debug()</code> logs (no passwords, tokens, OTPs)	
G4	No unnecessary re-renders (deps arrays correct, stable refs)	
G5	Lists paginated	
G6	Images sized + lazy-loaded where appropriate	
G7	Lazy-loaded subtrees use <code><Suspense></code> with a <code>Loader</code> fallback	

H. Accessibility

#	Check	P/F/NA
H1	Semantic HTML elements used	
H2	<code>alt</code> on every <code></code> (<code>alt=""</code> for decorative)	
H3	<code>aria-label</code> on icon-only buttons / links	
H4	Keyboard-only operability verified	
H5	Visible focus rings preserved	
H6	<code>aria-invalid</code> + <code>role="alert"</code> on error rows	
H7	Color contrast meets AA on used token pairings	

I. Debug logs & wiki

#	Check	P/F/NA
I1	<code>debug(scope)</code> logs present at every async / branch / dispatch / calc	
I2	Raw <code>console.log</code> calls follow the same <code>[scope]</code> prefix	
I3	No sensitive data in any log	
I4	<code>wiki/components.md</code> (or relevant page) updated	
I5	<code>wiki/log.md</code> entry appended with date + summary	
I6	Confidence tags accurate; demoted to <code>□ STALE</code> if changed under foot	

J. Tests

#	Check	P/F/NA
J1	New route has at least one Playwright spec	
J2	Specs use <code>getByRole</code> / <code>getByLabel</code> (no brittle CSS selectors)	
J3	Specs pass locally and (if CI is wired) in CI	
J4	Author quoted the verification output in the PR description	

18.3 Review-Comment Format

```
[FAIL – B2] File: src/components/cards/EntryMethodCard.jsx, Line 24
ISSUE: Class names concatenated with template literals instead of classNames()
EXPECTED: classNames(baseStyles, variants[variant], className)
ACTUAL:   `${baseStyles} ${variant === 'active' ? 'bg-brand-green' : ''}`
ACTION:   Refactor to use classNames() per SOP §5.2.
```

19. Figma Fidelity & Design Reference

19.1 The Rule

Every Figma spec value must be implemented exactly as seen. No rounding, no "close enough", no substitutions. Applies to: typography (size, weight, family, line-height, letter-spacing), spacing (padding, margin, gap), sizing (width, height, min/max), layout (display, alignment, flex/grid), visuals (border-radius, border, background, opacity, box-shadow).

19.2 Fluid Sizing with `clamp()`

All scalable properties must use `clamp()` to interpolate between mobile and desktop reference values. **Do not** hardcode breakpoint-static px with media queries for sizing.

Formula:

```
preferred = min_px + (max_px - min_px) * (100vw - MIN_VW) / (MAX_VW - MIN_VW)
```

Default `MIN_VW = 390`, `MAX_VW = 1440`. Example for `14 px @ 390 → 24 px @ 1440`:

```
font-size: clamp(0.875rem, 0.399rem + 1.905vw, 1.5rem);
```

Helper (use as inline-style values or in `cssText`):

```
const fluid = (minPx, maxPx, minVw = 390, maxVw = 1440) => {
  const minRem = minPx / 16;
  const maxRem = maxPx / 16;
  const slope = (maxPx - minPx) / (maxVw - minVw);
  const intercept = minPx - slope * minVw;
  const interceptRem = intercept / 16;
  const slopeVw = slope * 100;
  return `clamp(${minRem}rem, ${interceptRem}rem + ${slopeVw}vw, ${maxRem}rem)`;
};

// Usage:
style={{ fontSize: fluid(14, 24), padding: fluid(8, 16) }}
```

Reserve `@media` for **layout changes only** (flex-direction, grid column count, show/hide, navigation collapse) — not for sizing.

19.3 Figma MCP Workflow (CRITICAL)

Always use node-detail tools. Never screenshot-guess.

1. Get the node-id from wiki/figma-node-map.md or the Figma URL.
2. `get_design_context(nodeId)` → returns CSS properties (font-size, weight, fill, padding, ...).
3. Read the exact values.
4. Implement in JSX/CSS using `clamp()` where applicable.
5. Take a Playwright MCP screenshot at 390 / 768 / 1440 to visually verify.

For hero-style frames with **free-floating decoration** (orbs, scribbles, sparkles),

`get_design_context` flattens the absolute geometry into auto-layout approximations and may drop rotation. Workflow:

1. `get_metadata` first — raw geometry (x, y, width, height, `relativeTransform`).
2. Cross-validate with `get_design_context` + `get_variable_defs` + `get_screenshot`. Save outputs to a temp file if the frame is large.
3. Keep selections under ~12,000 tokens (subtrees, not whole frames).
4. Round-trip decorative SVGs as exported assets in `src/assets/` — don't ask the LLM to redraw the path.
5. Use `src/utils/figmaPosition.js` to convert metadata into Tailwind / inline-style numbers (`relativeTo`, `rotationFromMatrix`, `relativePercent`).

Rotation gotcha: Figma stores CCW; CSS `rotate()` is CW. The helper flips the sign — **don't double-negate**.

19.4 Figma File & Node Map

- File: "*Gh Design system - onboarding*" — file key `Bin8roWL8sloyc36IgFMuT`
- Top-level URL: `https://www.figma.com/design/Bin8roWL8sloyc36IgFMuT`
- Per-frame node IDs: `wiki/figma-node-map.md` (populate as designs land; demote to `□ STALE` when frames are restructured).

To open a specific frame in Figma, construct:

```
https://www.figma.com/design/Bin8roWL8sloyc36IgFMuT?node-id=NODE_ID
```

(Replace `:` with `-` in the URL, or URL-encode it as `%3A`.)

19.5 Generating a Figma Personal Access Token

For programmatic access (scripts, image exports):

1. Open Figma → click the Figma icon (top-left) → **Help and account**.
2. **Account settings** → **Security** tab.
3. Scroll to **Personal access tokens** → **Generate new token**.
4. Name it descriptively (e.g. `talenthub-dev-token`).
5. Scopes: `file_content:read` at minimum; optionally `file_content:write` for posting comments.
6. **Copy the token immediately** — Figma shows it only once. Store in your password manager or a local `.env` (never committed) as `FIGMA_TOKEN`.

Curl example:

```
curl -H "X-Figma-Token: $FIGMA_TOKEN" \
  "https://api.figma.com/v1/files/Bin8roWL8sloyc36IgFMuT/nodes?ids=2849:66712&depth=3"
```

Without `file_content:read`, `/v1/files/` and `/v1/images/` return 403 *Invalid scope(s)*. Regenerate with the right scope if so.



20. Appendices

Appendix A – Quick Reference: Tailwind Utilities

Colors	bg-{brand-green accent neutral black yellow}-{light hover active dark dark-hover dark-active darker}
	bg-{informative success warning danger}-{...same ladder...}
	text-{content-primary content-secondary content-tertiary}
	bg-{background-default background-subtle}
	border-{border-default border-neutral}
Typography	font-sans font-display
	text-display-xl text-display-lg text-display-md text-display-sm text-display-italic
	text-{strong medium normal}-{25 50 75 100 200 300 400 500 600}
Radius	rounded-sm (6px) rounded-md (10px) rounded-lg (16px) rounded-xl (24px) rounded-pill (100px)
Spacing	p-1 (4px) ... p-2 (8px) ... p-4 (16px) ... p-8 (32px) ... p-12 (48px) ... p-20 (80px)
Shadows	shadow-sm shadow-md shadow-lg shadow-green
	shadow-top-{100..400} shadow-bottom-{100..400} shadow-button-shelf
Layout	mx-auto w-full max-w-[1728px] (page column)
	transition-all duration-300 ease-in (interactive elements)

Package	Import	Common usage
<code>axios</code>	<code>import { api } from '../services/api.js'</code>	HTTP — always use the configured instance
<code>universal-cookie</code>	<code>import Cookies from 'universal-cookie'</code>	Cookie read/write
<code>redux-persist</code>	<code>import { persistStore, persistReducer } from 'redux-persist'</code>	State persistence
<code>react-hook-form</code>	<code>import { useForm } from 'react-hook-form'</code>	Form state
<code>zod</code> + <code>@hookform/resolvers</code>	<code>import { zodResolver } from '@hookform/resolvers/zod'; import { z } from 'zod'</code>	Schema validation
<code>@playwright/test</code>	<code>import { test, expect } from '@playwright/test'</code>	E2E tests

Appendix D — Glossary

Term	Meaning
Component	A reusable piece of UI. A React function that returns JSX.
Props	Data passed into a component from its parent.
State	Data that can change over time in a component or in Redux.
Slice	A self-contained chunk of Redux state with its reducers and actions.
Thunk	An async function that can dispatch Redux actions (used for API calls).
Interceptor	axios middleware that runs before every request or after every response.
Hook	A React function (prefix <code>use</code>) that gives access to state and lifecycle features.
<code>forwardRef</code>	A React wrapper that lets a parent get a ref to a child's DOM element.
Tailwind utility	A small CSS class that does one thing (e.g. <code>bg-brand-green</code>).
CSS custom property	A variable defined in CSS (e.g. <code>--color-brand-green: #387440</code>).
Variant	A visual variation of a component (e.g. primary vs secondary).
Token	A named design value (color, type size, spacing) defined once and reused everywhere.
<code>@theme</code>	Tailwind v4's in-CSS block where tokens are declared.
<code>clamp()</code>	CSS function that interpolates a value between a min and max relative to viewport width.
MCP	Model Context Protocol — how Claude Code connects to external tools (Playwright, Figma).
Figma node ID	A unique identifier for any element in a Figma design file.
Layout shell	A wrapper component (Navbar + Outlet + Footer) that surrounds page content.
Outlet	React Router's placeholder for child-route content inside a layout.

Term	Meaning
Confidence tag	A marker on a wiki claim indicating how reliable it is.
Wiki	The persistent, compounding knowledge base in <code>wiki/</code> .

| Appendix E — Onboarding Checklist for New Devs

Day 1 — environment

- ☐ Clone the repo, `npm install`, copy